2 Bayesian Computation

The content of this chapter draws heavily on *Bayesian Analysis with Python: Unleash the power and flexibility of the Bayesian framework* by Osvaldo Martin.

In Bayesian statistics, we update prior beliefs using data via Bayes' theorem to obtain the posterior distribution. However, the posterior often involves complicated integrals that cannot be solved exactly. Bayesian computation provides ways to approximate these posteriors using numerical techniques.

- Bayesian statistics is conceptually straightforward. We begin with data that has already been observed, and therefore cannot be changed. The focus is on the unknown parameters that we wish to learn from this data. Since the exact values of these parameters are uncertain, we describe this uncertainty using probability. In this framework, probability is used as a way to represent our degree of belief about the possible values of the parameters, given the observed data.
- If a quantity is unknown, we represent our uncertainty about it by assigning a probability distribution.
- This initial distribution reflects what we believe about the quantity **before seeing any data**, and is called the prior distribution.
- Once data is observed, Bayes' theorem is used to update this prior into a posterior distribution, which represents what we know **after taking the data into account**.
- In this sense, Bayesian statistics can be viewed as a process of learning from data, where our beliefs are updated as new information becomes available.
- Although the idea is conceptually simple, fully probabilistic models often result in expressions that cannot be solved analytically. For many years, this created practical difficulties, as exact solutions were not available. This challenge was one of the main reasons that slowed down the widespread use of Bayesian methods.
- There are several methods to compute the posterior even when it is not possible to solve it analytically. Some of the methods are:

Non-Markovian methods:

- Grid computing
- Quadratic approximation

- Variational methods

Markovian methods:

- Metropolis-Hastings
- Hamiltonian Monte Carlo/No U-Turn Sampler

2.1 Non-Markovian methods

- Non-Markovian methods estimate posterior distributions without using Markov chains.
- They are generally faster because they do not rely on iterative sample transitions.
- They can give good results for some problems, but for complex cases they often provide only approximate solutions. These approximations are still useful as starting points for more accurate Markovian methods, which will be discussed later.

2.1.1 Grid computing

- Grid computing is a simple brute-force method for approximating the posterior distribution.
- First, a reasonable interval for the parameter is chosen based on the prior.
- Then, a set of equally spaced grid points is created over this interval.
- For each point, the prior and likelihood are computed and multiplied to obtain an unnormalized posterior value. These values can optionally be normalized by dividing each by the total sum.
- Using more grid points improves the accuracy of the approximation, and with infinitely many points, the exact posterior can be obtained.
- However, this method does not scale well to multiple parameters.
- As the number of dimensions increases, most grid points contribute very little to the posterior, making the approach computationally inefficient for high-dimensional problems.

Chapter 2 Python Lab – Exercise 1 implements the grid approach to solve the coin-flipping problem

Refer Chapter2 Python Lab - Exercise 1

2.1.2 Quadratic (Laplace) Approximation

- The quadratic approximation, also known as the Laplace or normal approximation, is a method used to approximate a posterior distribution with a Gaussian distribution.
- It is based on the idea that the shape of the posterior near its peak (mode) can often be well approximated by a parabola, which corresponds to a normal distribution.
- The method involves two main steps.
  - First, the mode of the posterior distribution (also called the MAP estimate. (Maximum A Posteriori) which means the value of the parameter that maximizes the posterior distribution) is identified using an optimization method. This value is taken as the mean of the approximating Gaussian distribution.
  - Second, the curvature of the posterior around the mode is estimated using the second derivative (or Hessian in multivariate cases). This curvature determines the variance (or standard deviation) of the Gaussian approximation.
- The second derivative measures how quickly the slope of a function changes around a point. This rate of change of slope reflects the curvature of the function.
- If the second derivative is large in magnitude (very negative for a maximum), the function bends sharply around the peak -> high curvature -> narrow peak.
- If the second derivative is small in magnitude, the function bends slowly -> low curvature -> flatter peak.
- The Laplace approximation works well when the posterior is unimodal and approximately symmetric around the mode. In such cases, the Gaussian approximation closely matches the true posterior near its peak.
- However, if the posterior is highly skewed, has multiple modes, or has strong non-linear structure, the approximation may be less accurate.
- Despite this limitation, it provides a simple and computationally efficient way to approximate posterior distributions, especially when exact inference is difficult.

2.1.3 Variational methods

- Exact Bayesian inference is often computationally expensive or intractable, especially for large datasets or complex models.
- Markov Chain Monte Carlo (MCMC) methods can also be slow and may not scale well.

- Variational methods provide a faster alternative by turning inference into an optimization problem. They are particularly useful for large-scale problems and can also provide good initial approximations for more accurate methods.
- Variational methods replace a complicated posterior distribution with a simpler one (such as a Gaussian). Instead of trying to compute or sample from the exact posterior, we search for the best simple distribution that approximates it as closely as possible.
- But note: we do not know the exact posterior distribution because it involves an intractable normalizing constant. However, we can compute the posterior up to a constant using:

$$p(\theta \mid y) \propto p(y \mid \theta)\, p(\theta)$$

- That is, we can evaluate the likelihood and the prior, which is enough for variational methods.

Key steps:

1. Choose a simple distribution

We select a family of distributions (e.g., Gaussian) to approximate the posterior. This defines the "shape" we are allowed to use.

2. Transform parameters (if needed)

Parameters with constraints (e.g., positive values) are transformed into an unconstrained space (such as the real line) to simplify optimization.

3. Define ELBO (objective function)

We define a function called the Evidence Lower Bound (ELBO), which measures how well the chosen distribution approximates the posterior using quantities we can compute (likelihood, prior, and the variational distribution).

4. Optimize the parameters

We adjust the parameters of the chosen distribution to maximize the ELBO. This indirectly makes the approximation closer to the true posterior.

5. Use the result

After optimization, the resulting distribution is used as an approximation of the posterior for inference.

- Both Laplace and variational methods aim to approximate the posterior with a simpler distribution (often Gaussian).

- Laplace approximation uses a local quadratic (second-order) approximation around the mode. It relies only on information near the peak of the posterior.
- Variational methods choose a family of distributions (e.g., Gaussian). They optimize the parameters of that distribution to best match the posterior. They use a global optimization objective (ELBO), not just local curvature.
- Key difference:
  - Laplace = local approximation at the mode.
  - Variational = global optimization over a chosen distribution family.
- However, variational methods are model-specific and require defining an appropriate approximation for each model.

2.2 Markovian methods

- MCMC (Markov Chain Monte Carlo) methods are used to approximate a posterior distribution by generating samples from it.
- Instead of computing the full posterior directly, these methods draw samples in such a way that the frequency of samples reflects the shape of the posterior.
- MCMC methods work by exploring the parameter space and spending more time in regions where the posterior probability is high. Regions with higher probability are visited more often, while regions with low probability are visited less often. Over time, the collection of samples represents the posterior distribution.
- These methods require the ability to compute the likelihood and the prior at a given point, but not the full posterior in closed form. As the number of samples increases, the approximation becomes more accurate.
- To understand what MCMC methods are we are going to split the method into the two MC parts, the Monte Carlo part and the Markov Chain part

2.2.1 Monte Carlo Methods

- Monte Carlo methods are techniques that use random sampling to approximate a value or solve a problem. The main idea is that instead of solving a problem exactly, we simulate many random samples and use them to estimate the result.
- These methods are useful when a problem is difficult to solve analytically but easy to simulate. By generating a large number of random samples, we can approximate quantities such as probabilities, integrals, or expectations.

- If random samples are drawn from a process, the proportion of samples that satisfy a condition can be used to estimate the desired quantity
- For example, in estimating π, random points are generated inside a square, and the proportion that falls inside an inscribed circle is used to approximate the value of π.

Refer Chapter2 Python Lab - Exercise 2

- In general, the accuracy of Monte Carlo methods improves as the number of samples increases.

2.2.2 Markov Chain

- In Bayesian inference, we want to sample from the posterior distribution $p(\theta|y)$, but we often cannot compute or sample from it directly.
- A Markov chain helps by generating a sequence of values of θ step by step:
  - Start from an initial value.
  - Move to a new value using a transition rule that depends only on the current value.
- The transition rule is designed using the likelihood and prior, so that:
  - Moves to high-probability regions are more likely.
  - Moves to low-probability regions are less likely.
- If the Markov chain is constructed properly (satisfying detailed balance), then, after many steps, the values visited by the chain behave like samples from the posterior
- Here "detailed balance" means the flow of probability between any two states is equal in both directions, which keeps the distribution stable and ensures the Markov chain samples from the correct distribution.
- This idea is called MCMC (Markov Chain Monte Carlo):
  - Markov Chain -> generates dependent steps
  - Monte Carlo -> uses these steps as samples to approximate the posterior
- A Markov chain is used to generate samples by moving step by step in a way that, over time, the collected values represent the posterior distribution
- MCMC constructs a Markov chain whose long-run behavior matches the target distribution (e.g., posterior).
- The most popular method that guarantees detailed balance is the Metropolis-Hasting algorithm

2.2.3 Metropolis-Hasting algorithm

- Metropolis–Hastings is a specific MCMC algorithm used to build the Markov chain.
- It works as follows:

1. Start at an initial value.
2. Propose a new value using a simple distribution (e.g., Gaussian noise).
3. Compute an acceptance probability based on how likely the new value is compared to the current one.
4. Accept or reject the proposal:
    - If accepted -> move to the new state
    - If rejected -> stay at the current state
5. Repeat many times to build a chain.

At the end of the procedure, we obtain a sequence of values, often called a sample chain or trace. If the algorithm has been applied correctly, this sequence provides an approximation to the posterior distribution

Refer Chapter2 Python Lab - Exercise 3